MovieTime: Blockchain-Powered Digital Entertainment and Subscription Infrastructure on BOT Chain

Whitepaper v1.0

1. Abstract

MovieTime is a subscription streaming platform whose catalog metadata, access rules, subscriptions and payment records are stored on BOT Chain. Users pay with the BOT native token or USDT, and access is determined by a blockchain timestamp rather than a private billing database. Video files are deliberately kept off-chain and referenced by a configurable external URL for content the publisher is licensed to distribute. The platform is built with HTML5, CSS3, vanilla JavaScript, ethers.js and Solidity, with no servers or databases.

2. Introduction

Streaming has become a primary way people consume entertainment, yet subscribers have little visibility into how their payments, access rights and data are handled. MovieTime applies public blockchain infrastructure to the parts of streaming where transparency and portability matter most: who may watch, what they paid, and when access ends.

3. Streaming Industry Challenges

Subscribers face fragmented accounts, opaque billing, difficult cancellation and regional payment friction, particularly where card access is limited. Publishers depend on platform-controlled accounting and slow settlement. Platforms carry the cost of centralized billing, chargebacks and account databases. Piracy and credential sharing remain persistent problems that no single design fully eliminates.

4. Product Vision

A premium, Netflix-quality experience in which a wallet is the account, a transaction is the receipt, and an expiry timestamp is the subscription. MovieTime aims to give audiences portable access and give publishers transparent revenue records, while remaining honest about what a blockchain can and cannot do for media delivery.

5. Proposed Solution

MovieTime combines three layers: a Solidity contract suite on BOT Chain that holds the catalog, categories, plans, subscriptions and payments; a static web application that reads and writes this state through ethers.js; and an external video host that serves licensed files over HTTPS. The onchain layer is the source of truth for entitlements; the video host enforces delivery.

6. Platform Architecture

The frontend is a static site with no backend. On connection it validates the BOT Chain network id, reads the user's profile and expiry, and loads the published catalog. The catalog supports movies, series, seasons, episodes, documentaries, short films and trailers, linked by parent identifiers. A custom HTML5 player provides play/pause, volume, seeking, fullscreen, subtitles, quality selection and continue-watching. Playback position is kept in browser storage as a convenience only; it is not permanent application data.

7. Content Management

Authorized administrators add and edit titles, publish or unpublish them, feature content and manage categories through the dashboard. Each title stores title, description, poster and banner references, genre, year, runtime, rating, cast, premium or free status and an external video reference. Video references may list multiple qualities. Large video files are never stored on-chain and are never represented as on-chain. Storage limitation: a blockchain is inefficient for video-scale data, so only references and access records are recorded. Publishers must hold distribution rights for every title they list.

8. Subscription Model

Four plans are supported: weekly, monthly, quarterly and yearly. Administrators set the duration and price of each plan in BOT and USDT. A purchase extends the user's expiry from the later of the current expiry or the current block timestamp. Renewal is an explicit new purchase by the user. The application displays the current plan, price, currency, start date, expiry date and full payment history.

9. Payment Architecture

BOT payments are sent with the subscribe transaction and must equal the plan price exactly; funds are forwarded to the treasury address in the same transaction. USDT payments use a standard ERC-20 approval for the exact plan price followed by a transferFrom to the treasury. No recurring or automatic deduction exists: the contract never pulls funds without a transaction signed by the user for that payment. Every payment is recorded with payer, plan, currency, amount, start, expiry and time.

10. Smart Contract Design

The contract keeps a title registry, a category registry, four plan slots, per-user expiry, profiles and an append-only payment ledger. Administrator functions manage titles, categories, plans and the treasury; the owner manages administrators. Reads are paginated. Reentrancy is guarded, payment state is updated before external transfers, and USDT transfers handle tokens that return no value. Expiry uses block.timestamp, which is suitable for durations measured in days.

11. Content Access Control

Premium video references are returned by the contract's getVideo function only to callers with an active subscription or administrator role. This gates the application layer. Contract storage is publicly readable through raw RPC calls, so a determined reader could extract a stored URL. For strong protection, publishers should serve video from a host that issues short-lived signed URLs or checks entitlement against the contract before delivery.

12. Security Considerations

Principal risks include administrator key compromise, mistaken price configuration, treasury misconfiguration, URL leakage, unlicensed content and malicious token behaviour. Mitigations include multi-signature wallets for the owner and treasury, minimal and exact approvals, public events for all state changes, independent audit before mainnet use, and a clear takedown process for rights holders. The frontend escapes all onchain text before rendering.

13. Business Model

Revenue is subscription income paid directly to the treasury. Possible extensions include revenue sharing with rights holders through a splitter contract, gift subscriptions, bundled partner plans and regional pricing. Costs include video hosting and delivery, content licensing and security review.

14. Roadmap

Phase 1: catalog, subscriptions, wallet integration and administrator dashboard on BOT Chain testnet. Phase 2: security audit, signed-URL delivery gateway and mainnet launch. Phase 3: publisher revenue sharing, multi-admin roles and subtitle and audio-track management. Phase 4: mobile web optimisation, gifting and partner bundles.

15. Conclusion

MovieTime shows that subscriptions, catalog records and receipts can be transparent and user-owned on BOT Chain while video delivery remains in a purpose-built off-chain layer. By being explicit about this boundary and requiring a signature for every payment, the platform offers a practical and trustworthy foundation for blockchain-powered digital entertainment.